Министерство образования Республики Беларусь

Учреждение образования

«Брестский Государственный технический университет»

Кафедра ИИТ

Лабораторная работа №2

По дисциплине «Современные методы защиты компьютерных систем»

Тема: «Защита веб-периметра и ограничение интенсивности сетевого трафика средствами межсетевых экранов прикладного уровня»

Выполнил:

Студент 4 курса

Группы ИИ-25

Бекназарова А. С.

Проверил:

Хацкевич А. С.

Брест 2026

Цель работы: практическое освоение методов развертывания межсетевых экранов прикладного уровня (WAF), написания пользовательских правил фильтрации L7-угроз, а также проектирование механизмов защиты от DDoS-атак и ограничения частоты запросов (Rate Limiting) на базе обратного прокси-сервера Nginx

Вариант 3

WAF: Блокируемый User-Agent - DirBuster-1.0-RC1**, ID правила WAF –** 10003, **Rate Limit: Эндпоинт (Target) -** /api/register**, Rate Limit: rate -** 1r/s**, Rate Limit: burst -** 3

Ход работы

Задание 1: базовая фильтрация угроз через WAF (OWASP Core Rule Set).

Взлом:

```
curl.exe -i "http://127.0.0.1:5000/?id=1%27%20OR%20%271%27%3D%271"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.9 Python/3.10.22
Date: Tue, 03 Nov 2026 11:22:30 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 31
Connection: close

Backend OK. Endpoint reached: /
```

Исправление:

```
curl.exe -i "http://127.0.0.1/?id=1%27%20OR%20%271%27%3D%271"
HTTP/1.1 403 Forbidden
Server: nginx/1.30.6
Date: Tue, 03 Nov 2026 11:22:56 GMT
Content-Type: text/html
Content-Length: 153
Connection: keep-alive

<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.30.6</center>
</body>
</html>
```

Задание 2: написание пользовательских правил фильтрации (L7).

Результат:

```
curl.exe -i -H "User-Agent: DirBuster-1.0-RC1" http://127.0.0.1/
HTTP/1.1 403 Forbidden
Server: nginx/1.30.6
Date: Tue, 03 Nov 2026 11:25:37 GMT
```

```
Content-Type: text/html
Content-Length: 153
Connection: keep-alive

<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.30.6</center>
</body>
</html>
```

Проверка:

```
docker-compose logs waf | findstr /I "10003"
WARN[0000] D:\Учёба\4 курс\СМЗКС\2\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
waf-1  | 2026/11/03 11:25:37 [error] 567#567: *2 [client 172.18.0.1] ModSecurity: Access denied with code 403 (phase 1). Matched "Operator `Contains' with parameter `DirBuster-1.0-RC1' against variable `REQUEST_HEADERS:User-Agent' (Value: `DirBuster-1.0-RC1' ) [file "/etc/modsecurity.d/owasp-crs/rules/custom.conf"] [line "2"] [id "10003"] [rev ""] [msg "Blocked Custom Scanner DirBuster"] [data ""] [severity "0"] [ver ""] [maturity "0"] [accuracy "0"] [tag "modsecurity"] [hostname "127.0.0.1"] [uri "/"] [unique_id "17937051372.881971"] [ref "o0,17v55,17"], client: 172.18.0.1, server: localhost, request: "GET / HTTP/1.1", host: "127.0.0.1"
```

Задание 3: Защита от L7 DDoS-атак (Rate Limiting).

Стресс тест:

```
.\bombardier.exe -n 500 -c 10 http://127.0.0.1/api/register
Bombarding http://127.0.0.1/api/register with 500 request(s) using 10 connection(s)
 500 / 500 [============================================] 100.00% 2482/s 0s
Done!
Statistics        Avg      Stdev        Max
  Reqs/sec      2547.69     819.23    3274.04
  Latency        3.80ms     2.53ms    25.66ms
  HTTP codes:
    1xx - 0, 2xx - 4, 3xx - 0, 4xx - 0, 5xx - 496
    others - 0
  Throughput:     1.06MB/s
```

До применения конфигурации все 500 запросов обрабатывались успешно. После добавления директив rate=1r/s и burst=3 496 запросов были заблокированы WAF с ошибкой 503 (5xx), что подтверждает работоспособность защиты

user_nginx.conf

```
# RATE LIMITING (Задание 3)
limit_req_zone $binary_remote_addr zone=mylimit:10m rate=1r/s;

server {
    listen 80;
    server_name localhost;

    location /api/register {
        limit_req zone=mylimit burst=3 nodelay;
```

```
        proxy_pass ${BACKEND};
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    location / {
        proxy_pass ${BACKEND};
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

1. Защита от атаки «Slowloris» (Slow HTTP Attack)

Атака Slowloris — это тип DDoS-атаки прикладного уровня (L7), при которой злоумышленник открывает множество TCP-соединений с сервером и отправляет HTTP-заголовки очень медленно (по одному байту), не завершая запрос. Сервер вынужден держать каждое такое соединение открытым, ожидая окончания заголовков, что быстро исчерпывает пул доступных воркеров (worker connections) и делает сервер недоступным для легитимных пользователей.

Для защиты от данной атаки на уровне обратного прокси-сервера Nginx применяются следующие директивы:

- client_header_timeout 10s; — ограничивает максимальное время чтения заголовка запроса от клиента. Значение 10 секунд оптимально: обычный клиент отправляет заголовки за миллисекунды, а 10 секунд с запасом хватает для медленных мобильных соединений. Однако злоумышленник не сможет удерживать соединение дольше 10 секунд, после чего Nginx его принудительно разорвет.
- client_body_timeout 10s; — ограничивает время на чтение тела запроса (актуально для медленных POST-запросов). 10 секунд достаточно для передачи легитимных данных и блокирует попытки удержания соединения через медленную отправку тела.
- keepalive_timeout 15s; — определяет время, в течение которого сервер будет держать открытым idle-соединение (бездействующее). Если поставить слишком много (например, 60 секунд), злоумышленник сможет накапливать тысячи "висящих" соединений. Если слишком мало (1-2 секунды) — легитимные клиенты будут вынуждены постоянно переподключаться, что создаст лишнюю нагрузку. 15 секунд — оптимальный баланс.
- (Дополнительно можно добавить) send_timeout 10s; — ограничивает время передачи ответа клиенту, который перестал читать данные.

2. Эшелонированная архитектура защиты CDN + WAF

DDoS-атаки различаются по уровню модели OSI. Для защиты от объемных атак (L3/L4) и прикладных атак (L7) применяется эшелонированная (многоуровневая) архитектура.

Схема обработки трафика:

Пользователь -> CDN (L3/L4 защита) -> WAF (L7 защита) -> Сервер приложения

Описание работы:

1. **CDN (Content Delivery Network)**. Служит для защиты от объемных атак (UDP-flood, SYN-flood). Запросы пользователей попадают не на основной сервер, а на ближайший

узел CDN (технология Anycast). Благодаря глобальной распределенности, вредоносный трафик "размазывается" по сотням серверов по всему миру, и его интенсивность на каждый узел падает. CDN осуществляет фильтрацию сетевых атак (scrubbing) и кэширование статики. До основного сервера доходит только очищенный трафик.

2. **WAF (Web Application Firewall)**. После CDN трафик уже не объемный, но внутри HTTP-запросов могут скрываться прикладные атаки (SQL-инъекции, XSS, RCE). WAF анализирует каждый запрос на 7-м уровне (URL, заголовки, тело), применяет правила OWASP CRS и пользовательские правила (например, блокировка сканеров), а также ограничивает частоту запросов (Rate Limiting), защищая от брутфорса и L7 DDoS.
3. **Сервер приложения**. Получает только те запросы, которые прошли все фильтры. Это радикально снижает нагрузку на сервер и полностью исключает возможность успешного проведения вредоносных запросов к уязвимому коду приложения.

Вывод: в данной лабораторной работе я освоила методы развертывания межсетевых экранов прикладного уровня (WAF), написания пользовательских правил фильтрации L7-угроз, а также проектирование механизмов защиты от DDoS-атак и ограничения частоты запросов (Rate Limiting) на базе обратного прокси-сервера Nginx.